

Sastrería Andrés Chimunja — Backend

API REST (Node.js + Express + Knex + PostgreSQL) para el sistema de gestión de la sastrería.

Requisitos

- Node.js 18+
- PostgreSQL 14+
- PM2 (`npm i -g pm2`) para producción

Instalación local

```
npm install
cp .env.production .env # o crea tu propio .env de desarrollo (ver variables abajo)
npm run migrate
npm run seed-admin      # crea el usuario Admin inicial
npm run dev             # http://localhost:3000, con nodemon
```

Variables de entorno

Variable	Descripción
<code>NODE_ENV</code>	<code>development</code> o <code>production</code> . Determina qué bloque de <code>knexfile.js</code> se usa.
<code>PORT</code>	Puerto donde escucha Express (default <code>3000</code>).
<code>DB_HOST</code> , <code>DB_PORT</code> , <code>DB_USER</code> , <code>DB_PASSWORD</code> , <code>DB_NAME</code>	Conexión a PostgreSQL.
<code>DB_SSL</code>	<code>true</code> solo si el proveedor de PostgreSQL exige SSL. Si la DB corre en el mismo VPS, déjalo en <code>false</code> .
<code>JWT_SECRET</code>	Secreto para firmar tokens JWT. Debe ser un valor largo y aleatorio en producción (nunca reutilizar el de desarrollo).
<code>FRONTEND_ORIGIN</code>	Dominio(s) permitido(s) por CORS, separados por coma (ej. <code>https://sastreria.com</code>). Sin esto, CORS cae al valor por defecto <code>http://localhost:4200</code> .
<code>WHATSAPP_TOKEN</code> , <code>WHATSAPP_PHONE_ID</code>	Credenciales de WhatsApp Meta Cloud API para envío de recibos.

En `.env.production` ya están los valores fijos (incluyendo un `JWT_SECRET` fuerte generado). Los campos marcados `CAMBIAR_*` deben completarse con los datos reales del VPS antes de desplegar.

Despliegue en VPS

1. Clonar el repo y correr `npm install --omit=dev`.
2. Copiar `.env.production` al VPS y completar los placeholders `CAMBIAR_*` (host, usuario y password de la DB, `FRONTEND_ORIGIN`).
3. Crear la base de datos en PostgreSQL y correr las migraciones:

```
NODE_ENV=production npm run migrate
```

4. Levantar el proceso con PM2:

```
npm run pm2:start      # pm2 start ecosystem.config.js --env production
pm2 save               # persiste el proceso entre reinicios del VPS
pm2 startup            # configura el arranque automático del sistema
```

5. Verificar que responde: `curl http://localhost:3000/health` → `{"status":"ok"}`.

Comandos PM2 útiles

```
npm run pm2:restart    # reinicia el proceso tras un deploy
npm run pm2:logs       # ver logs en vivo
pm2 status             # ver estado del proceso
```

Actualizar el backend tras un cambio de código

```
git pull
npm install --omit=dev
NODE_ENV=production npm run migrate
npm run pm2:restart
```

Scripts

Script	Uso
<code>npm run dev</code>	Desarrollo local con recarga automática (nodemon).
<code>npm start</code>	Arranque directo con <code>node</code> (sin recarga).
<code>npm run migrate</code>	Corre las migraciones pendientes.

Script	Uso
<code>npm run rollback</code>	Revierte la última tanda de migraciones.
<code>npm run reset-db</code>	Revierte todas las migraciones y las vuelve a aplicar (⚠ borra datos).
<code>npm run make-migration -- nombre_migracion</code>	Crea un nuevo archivo de migración.
<code>npm run seed-admin</code>	Crea el usuario Admin inicial.

Notas de seguridad

- `.env` y `.env.production` están en `.gitignore` — nunca deben commitearse.
- CORS solo permite los orígenes listados en `FRONTEND_ORIGIN`.
- Las imágenes subidas (`uploads/`) se clasifican solo en carpetas permitidas (`itempedido` , `medida`); si se agrega un nuevo tipo de referencia para imágenes, hay que sumarlo a `TIPOS_REFERENCIA_PERMITIDOS` en `src/config/multer.js`.